BANKING API KNOWLEDGE BASE

Login and Registration Questions & Answers

A practical reference to the current FastAPI authentication flow, written for developers, testers, and support conversations.

5	JSON + multipart	JWT + rotation
AUTH ENDPOINTS	REGISTRATION FORMATS	SESSION MODEL

Scope: current application behavior on the AI_Chatbot branch
Prepared 24 August 2026

SECTION 01

At a glance

The public authentication surface and the contract clients should follow.

Endpoint	Input	Successful result
POST /auth/register	JSON or multipart form	Access and refresh tokens
POST /auth/login	OAuth2 form: username + password	Access and refresh tokens
POST /auth/refresh	JSON: {"token": "..."}	Rotated token pair
POST /auth/logout	JSON: {"token": "..."}	Stored refresh token removed
POST /auth/google	JSON: {"id_token": "..."}	Access and refresh tokens

Important client detail
The login route uses the OAuth2 password form. The field is named **username**, but the application interprets its value as the customer's email address.

01

What happens after registration or login succeeds?

The API returns an access token, a refresh token, and **token_type: bearer**. The client sends the access token to protected routes in the **Authorization: Bearer <token>** header and retains the refresh token for session renewal.

02

Does the API use server-side sessions?

Not for access tokens. Access is represented by a signed JWT. Refresh tokens are also JWTs, but they are stored in PostgreSQL so they can be rotated and removed during logout.

03

Which data stores participate in authentication?

PostgreSQL stores users and refresh tokens. Valkey/Redis tracks login request counts for rate limiting. Profile images use local storage in development or S3 when the storage backend is configured as **s3**.

SECTION 02

Registration

Questions about creating a customer identity, validating input, and handling optional profile images.

04

How do I register a user?

Send **POST /auth/register**. Use **application/json** when no image is required, or **multipart/form-data** when uploading a profile image. A successful request currently returns HTTP 200 with a token pair.

05

Which registration fields are required?

The required fields are **email**, **first_name**, **last_name**, and **password**. Optional customer details are phone, address, date of birth, postcode, country, and city.

06

What format should date of birth use?

The schema expects a date value. In JSON or a multipart field, use the ISO date form **YYYY-MM-DD**, for example **1992-04-18**.

07

How are email addresses and names normalized?

Emails are trimmed and converted to lowercase. First name, last name, phone, address, postcode, country, and city are trimmed. First and last names must still contain at least one character after trimming.

08

What are the password rules?

A registration password must be 12 to 72 characters and contain at least one lowercase letter, one uppercase letter, one number, and one special character. Spaces do not count as special characters.

09

Can a customer register the same email twice?

No. The service checks for an existing user, and the database also makes email unique. A duplicate registration returns HTTP 409 with **Email already registered**.

10

Are passwords stored in plain text?

No. Registration hashes the password with bcrypt through Passlib before saving the user. The original password is not stored.

SECTION 03

Registration images and atomicity

How profile image validation works and what happens when part of registration fails.

11

Is a profile image required?

No. It is optional and is accepted only through multipart registration under the field name **profile_image**. JSON registration remains supported without an image.

12

Which image types are accepted?

The image contents must be JPEG, PNG, or WebP. The service validates the actual image format rather than relying only on the uploaded filename or content type.

13

What are the profile image limits?

The uploaded file may be at most 5 MB and at most 20 million pixels. Accepted images are resized to fit within 1024 by 1024 pixels, converted to RGB, and stored as WebP.

14

What happens when an uploaded image is invalid?

An invalid, empty, unsupported, or excessively large-dimension image returns HTTP 400. A file larger than 5 MB returns HTTP 413. Registration is not completed in either case.

15

What happens if database commit fails after the image was saved?

The database transaction rolls back. Because file storage cannot join the SQL transaction, the service performs compensating cleanup by deleting the saved image. Cleanup failure is logged without hiding the original registration failure.

16

Why does registration use a unit of work?

User creation, optional profile-image key assignment, and refresh-token creation must commit as one PostgreSQL transaction. If any database step fails, none of those database changes should remain.

17

What content types are rejected?

Any registration content type other than JSON or multipart form data returns HTTP 415. Malformed JSON and schema validation failures return HTTP 422.

SECTION 04

Login and rate limiting

How credentials are submitted, verified, and protected from repeated attempts.

18

How do I submit login credentials?

Send **POST /auth/login** as **application/x-www-form-urlencoded**. Put the email address in the OAuth2 **username** field and the password in the **password** field. A JSON login body is not the route's current contract.

19

Are email addresses normalized during login?

Yes. The email entered through the OAuth2 username field is trimmed and lowercased before repository lookup.

20

How are credentials verified?

The service looks up the normalized email and verifies the supplied password against the stored bcrypt hash. A missing user, missing password hash, or incorrect password all produce the same response.

21

What response is returned for incorrect credentials?

The API returns HTTP 401 with **Invalid email or password** and a **WWW-Authenticate: Bearer** header. The generic response avoids revealing whether an email is registered.

22

What is the current login rate limit?

The route permits three login requests per client IP within a 60-second window. Further attempts return HTTP 429 and include a **Retry-After** header.

23

Does a successful login reset the rate limit?

No. The limiter runs before authentication and counts each login request, including successful ones. The count expires with the 60-second Redis key. This is current behavior, not a statement that it is the final policy.

24

Can a Google-only account use password login?

Not unless it later gains a password hash. If the stored password hash is missing, password login returns the same HTTP 401 response as other invalid credentials.

SECTION 05

Tokens, refresh, and logout

The lifecycle of access and refresh tokens in the current application.

25

What claims are included in an access token?

The signed JWT contains **sub** for the user ID, **exp** for expiry, and **type: access**. It is signed with HS256 using the configured application secret.

26

What claims are included in a refresh token?

A refresh JWT contains the user ID, expiry, **type: refresh**, and a unique **jti**. The complete refresh token is also stored in PostgreSQL.

27

How long do tokens last?

In the current configuration, access tokens expire after 1 minute and refresh tokens expire after 7 days. These short values are suitable for demonstration and should be reviewed as an explicit security policy before production.

28

How does refresh-token rotation work?

Send the refresh token to **POST /auth/refresh**. The API verifies the JWT type and expiry, confirms the token exists in PostgreSQL, deletes it, creates a new token pair, and commits those changes together. The consumed refresh token cannot be reused after a successful rotation.

29

How does logout work?

Send the refresh token to **POST /auth/logout**. If it exists, the API deletes it. If it is already absent, logout still succeeds, making the operation idempotent.

30

Does logout immediately revoke the access token?

No. Access tokens are not stored in the database, so an already issued access token remains valid until its expiry. Logout removes the refresh token and prevents that session from being renewed.

31

What makes a refresh token invalid?

It is rejected when its signature or expiry is invalid, its type is not refresh, its subject is missing or invalid, it is absent from PostgreSQL, it has expired according to the stored timestamp, or its user no longer exists. An undecodable JWT currently returns HTTP 401 with **Could not decode token**; the other refresh validation failures use the refresh-token error handler.

SECTION 06

Google sign-in

How external identity verification maps a Google identity onto a local banking user.

32

How does Google sign-in work?

The client obtains a Google ID token and sends it to **POST /auth/google**. The backend verifies it against the configured Google client ID, requires a verified email, then issues the application's normal access and refresh tokens.

33

What happens when no matching local user exists?

The service creates a local user using the verified Google email and available given and family names. If a given name is absent, the portion of the email before the at-sign is used as a fallback.

34

What happens when the email already belongs to a password account?

If that user has no linked Google subject, the service links the verified Google identity to the existing account and saves it. Subsequent Google sign-ins can look up the user by that subject.

35

When is a Google account conflict returned?

If the matched local account is already linked to a different Google subject, the API returns HTTP 409 rather than silently replacing the established identity link.

36

Is Google sign-in restricted to a company email domain?

No. The current service accepts any Google email that Google reports as verified. There is no active company-domain restriction in the current branch.

37

What happens when the Google ID token is invalid?

Google verification failure returns HTTP 401 with **Invalid Google ID token**. An unverified Google email also returns HTTP 401.

SECTION 07

Errors and implementation notes

A final checklist for clients, testers, and future security hardening.

Status	Typical meaning
400	Invalid profile image
401	Invalid login, access token, refresh token, or Google identity
409	Duplicate email or conflicting Google identity
413	Profile image exceeds 5 MB
415	Unsupported registration content type
422	Missing or invalid request fields
429	Login request limit exceeded
503	Profile image storage is unavailable

38

Which failures deliberately use generic messages?

Password login always uses **Invalid email or password** for unknown emails and incorrect passwords. Most refresh validation failures use **Invalid or expired refresh token**; an undecodable JWT currently uses **Could not decode token**. Standardizing that decoder response is a possible follow-up.

39

Which values must never be logged or committed?

Plain-text passwords, access tokens, refresh tokens, Google ID tokens, the JWT signing secret, AWS credentials, and production environment files must be treated as secrets.

40

What should be reviewed before production?

Review token lifetimes, hashing cost, refresh-token storage or hashing, HTTPS-only transport, secure client token storage, rate-limit policy and proxy-aware client identification, secret rotation, audit logging, account-status enforcement, and removal of stale tokens.

Document boundary

This FAQ describes the implementation present on the AI_Chatbot branch on 24 August 2026. It is a technical reference, not a substitute for a production security review or customer-facing policy.